Experiment No:4

Student Name: Sumanyu Singh
Branch: BE CSE
Semester: 6th
Subject Name: Full Stack-II
SubjectCode:23CSH-309

UID:23BCS10801
Section/Group:KRG-3B
Date of Performance:03/02/2026

Aim: To integrate centralized state management within the EcoTrack application using Redux Toolkit and to efficiently manage asynchronous data processing through Redux async thunks, including proper handling of loading and error states.

Objective:

- To set up and configure a Redux store in a React application using Redux Toolkit.
- To design and implement Redux slices for managing different parts of application state.
- To perform asynchronous operations using Redux async thunks.
- To handle various request states such as loading, successful response, and error conditions.
- To link React components with the Redux store using React-Redux hooks.
- To initiate asynchronous API calls from UI components through Redux actions.
- To generate filtered or derived data views from the Redux state without altering the original global store.
- To enhance the overall user experience by implementing refresh mechanisms and responsive async feedback.

Implementation/Code:

1. **store.js:-**

```
store.js > ...
import { configureStore } from "@reduxjs/toolkit";
import logsReducer from "./features/logsSlice";

export const store = configureStore({
  reducer: {
    logs: logsReducer
  }
});
```

2. **app.js :-**

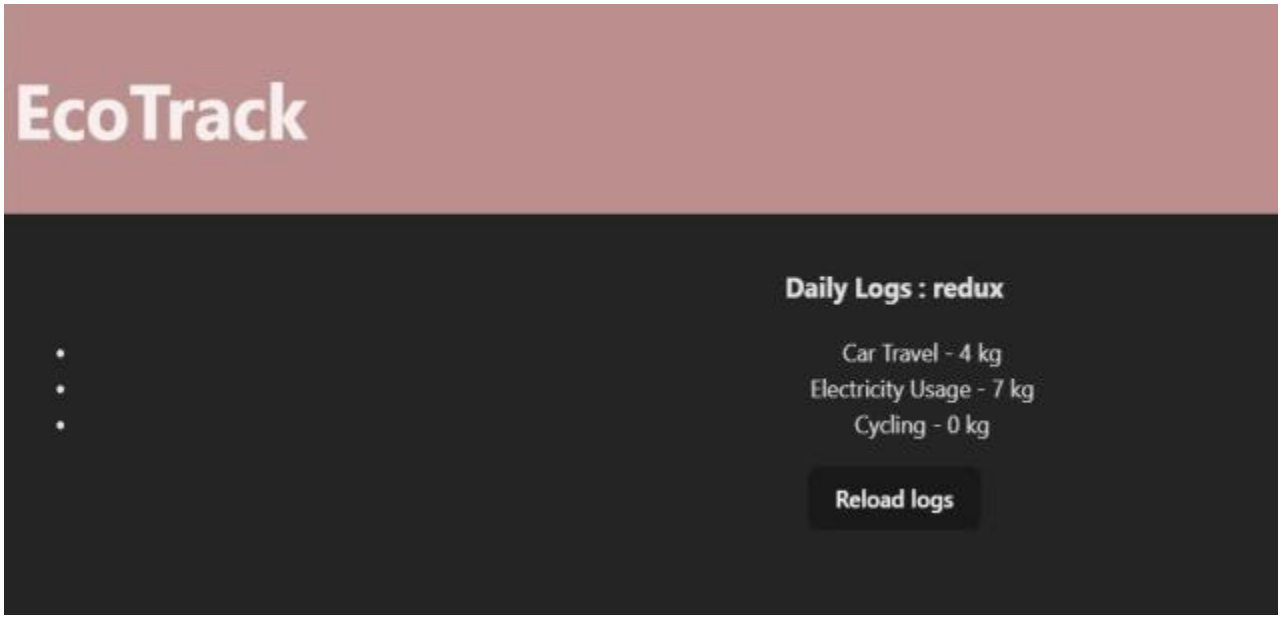
```
JS App.js > App
import React, { useEffect } from "react";
import { useDispatch, useSelector } from "react-redux";
import { fetchLogs } from "./features/logsSlice";

function App() {
  const dispatch = useDispatch();


  const { logs, status, error } = useSelector((state) => state.logs);


  useEffect(() => {
    dispatch(fetchLogs());
  }, [dispatch]);

  return (
    <div style={{ padding: "20px" }}>
      <h1>EcoTrack Dashboard</h1>

      {/* Loading State */}
      {status === "loading" && <p>Loading data...</p>}

      {/* Error State */}
      {status === "failed" && (
        <p style={{ color: "red" }}>Error: {error}</p>
      )}

      {/* Success State */}
      {status === "succeeded" && (
        <ul>
          {logs.map((log) => (
            <li key={log.id}>
              {log.activity} - {log.carbonFootprint} kg CO₂
            </li>
          ))}
        </ul>
      )}
```

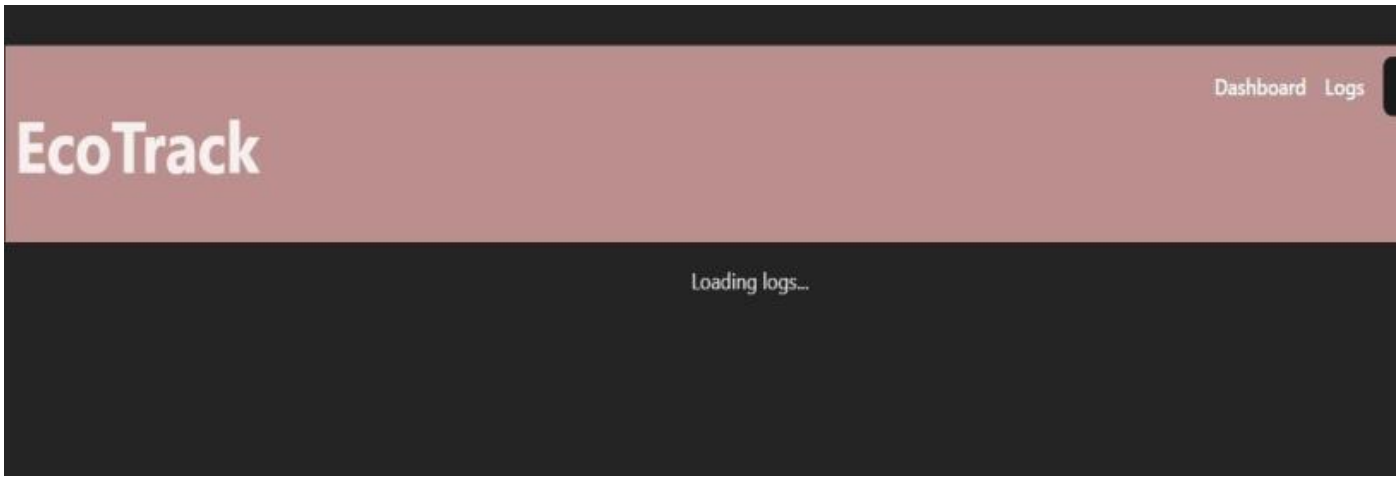
3. **logsslice.jsx:**

features > logsSlice.js > default

```javascript
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

// Async thunk for fetching data
export const fetchLogs = createAsyncThunk(
  "logs/fetchLogs",
  async () => {
    const response = await fetch("https://jsonplaceholder.typicode.com/posts");
    const data = await response.json();

    // Convert to EcoTrack format (example)
    return data.slice(0, 5).map((item) => ({
      id: item.id,
      activity: item.title,
      carbonFootprint: Math.floor(Math.random() * 100)
    }));
  }
);

const logsSlice = createSlice({
  name: "logs",
  initialState: {
    logs: [],
    status: "idle", // idle | loading | succeeded | failed
    error: null
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchLogs.pending, (state) => {
        state.status = "loading";
      })
      .addCase(fetchLogs.fulfilled, (state, action) => {
        state.status = "succeeded";
        state.logs = action.payload;
      })
      .addCase(fetchLogs.rejected, (state, action) => {
        state.status = "failed";
        state.error = action.error.message;
```

OUTPUT:





Learning Outcome :-

- Acquired practical knowledge of configuring a Redux store and organizing feature-based slices.
- Developed understanding of asynchronous data handling using Redux async thunks.
- Learned to manage multiple request states such as loading, success, and error within Redux.
- Gained experience in using selectors to compute derived data without modifying the global state.
- Improved skills in building scalable and maintainable React-Redux applications.